

AROMA

AI-powered Repository & Object Model Analyzer

IBM watsonx.ai Integration

Granite foundation model inference via REST API

IBM Bob Hackathon 2026 - Turn idea into impact faster
Built with IBM Bob IDE | Powered by IBM watsonx.ai Granite

Integration Overview

All five API routes share a consistent integration pattern through `src/lib/watsonx.ts`. The module handles IBM IAM token acquisition, caching, and automatic refresh, then calls the `watsonx.ai` chat completions endpoint with the Granite foundation model.

Authentication Flow

1. API route calls `callWatsonx()` or `analyzeWithWatsonx()`
2. `watsonx.ts` checks cached IAM token (valid 60 min)
3. If expired: POST `https://iam.cloud.ibm.com/identity/token`
4. Receives bearer token with `expires_in: 3600`
5. Token cached with 5-min refresh buffer
6. Authorized POST to `watsonx.ai` chat endpoint

Model Configuration Per Route

API Route	Temp	MaxTok	Rationale
<code>/api/analyze</code>	0.1	2000	Deterministic JSON extraction
<code>/api/chat</code>	0.3	1500	Slightly creative explanations
<code>/api/refactor</code>	0.1	2000	Consistent pattern detection
<code>/api/generate</code>	0.4	3000	More creative for docs prose
<code>/api/security</code>	0.05	2000	Highly deterministic findings

Response Parsing

`watsonx.ts` handles two response formats:

1. Chat completions: `choices[0].message.content` (current)
2. Text generation: `results[0].generated_text` (legacy)

Two JSON extraction helpers:

- `extractJSON`: Strips markdown fences, extracts JSON
- `safeParseJSON`: Parses JSON with fence stripping

All routes have static fallback data when AI is unavailable.

Verified Live Test Results

All 5 API routes tested with live IBM `watsonx.ai`:

- `/api/analyze`: Files, architecture, complexity OK
- `/api/chat`: 1245-char markdown flow analysis OK
- `/api/refactor`: 3 refactoring suggestions OK
- `/api/generate`: 3042-char README.md OK

- /api/security: SQL Injection + eval() detected, score 60/100